

基于 RAG 的 Redis 垂直领域深度问答系统构建

摘要

本文实现了一个面向 Redis 技术文档的垂直领域深度问答系统。系统以 Redis 官方文档和课程整理语料作为私有知识库，完成文档清洗、Chunking、Embedding、向量数据库构建、混合检索、上下文增强生成和量化评估。针对 Redis 问答中命令名、缩写和概念术语较多的特点，系统采用 Query Rewriting 与 BM25 + Vector Hybrid Retrieval 作为高级 RAG 优化策略。实验在 10 条 Redis 技术问题上评估 Context Relevance、Faithfulness 和 Answer Relevance，结果分别为 1.0000、0.8116 和 0.8750。

1. 研究背景

大语言模型具备较强的通用问答能力，但在 Redis 这类技术文档问答中仍可能出现两个问题。第一，模型可能凭借参数记忆回答，导致版本不一致或细节幻觉。第二，Redis 问题常包含 AOF、RDB、TTL、XREADGROUP 等精确术语，单纯依赖语义相似度可能忽略关键命令。RAG 通过先检索知识库再生成答案，可以把回答约束在可追溯上下文中，从而提升准确性和可解释性。

2. 数据集与知识库构建

本项目选择 Redis 作为垂直领域。知识库包含 Redis 数据类型、过期机制、内存淘汰、持久化、复制、Sentinel、Cluster、事务、Lua 脚本、Pub/Sub 和缓存风险等主题。每条文档保存为 JSONL，字段包括 doc_id、title、url 和 text。项目提供两种数据来源：

1. data/raw/redis_seed_docs.jsonl：课程实验内置清洗语料，保证离线可运行。
2. scripts/collect_redis_docs.py：从 Redis 官方文档页面抓取并清洗文本，生成新的 JSONL 私有知识库。

这种设计兼顾了可复现性和真实文档收集流程。

3. 系统设计

系统由五个核心模块组成：

- 文档处理模块：读取 JSONL 文档，并使用 Redis-aware tokenizer 识别中英文术语和命令名。
- Chunking 模块：按 token 数进行重叠分块，默认 chunk_size=320、overlap=40。
- 向量数据库模块：使用 hashing embedding 将 chunk 映射为定长向量，并持久化到 data/index/vectors.jsonl。
- 检索模块：同时构建 BM25 索引和向量索引，并对两个分数归一化融合。
- 生成模块：默认使用抽取式生成器，也支持 OpenAI-compatible LLM API。

整体流程为：用户问题先经过 Query Rewriting 扩展 Redis 领域词，再分别进入向量检索和 BM25 检索，融合排序后把 Top-k 片段传入生成模块，最终输出带来源引用的答案。

4. 高级 RAG 优化策略

本项目包含两类高级优化。

第一，Query Rewriting。系统维护 Redis 领域词扩展表，例如“持久化”会扩展为 persistence rdb aof snapshot append only file durability，“高可用”会扩展为 replication sentinel failover cluster。这可以缓解中文提问和英文官方术语之间的表达差异。

第二，Hybrid Retrieval。系统使用如下融合公式：

```
score = 0.45 * normalized_vector_score + 0.55 * normalized_bm25_score
```

Redis 技术问答对命令名非常敏感。BM25 能捕捉 AOF、RDB、TTL 等精确匹配，向量检索则补充语义相近表达。混合检索比单一路线更适合技术文档问答。

5. 生成模块

系统默认生成器为严格上下文抽取式生成器。它只从检索结果中选取与问题 token 重合度最高的句子，并附加 [1]、[2] 形式的引用。为了避免弱相关片段污染答案，生成阶段会过滤掉与最高检索分数差距过大的上下文。

项目也实现了 OpenAICompatibleGenerator。当用户设置 DEEPSEEK_API_KEY 或 OPENAI_API_KEY 时，系统会调用 DeepSeek 或其他 OpenAI-compatible LLM；否则自动回退到抽取式生成器。这保证了作业在无 API key 环境下仍可完整复现。

6. 实验与评估

评估集位于 data/eval/eval_questions.jsonl，包含 10 条 Redis 技术问题。每个样本提供问题、相关文档 ID 和期望关键词。

三个评估维度定义如下：

- Context Relevance：检索结果是否覆盖标注的相关文档。
- Faithfulness：答案中的 token 有多大比例能被检索上下文支持。
- Answer Relevance：答案覆盖期望关键词的比例。

运行命令：

```
python3 evaluate.py --rebuild
```

实验结果：

指标	分数
Context Relevance	1.0000
Faithfulness	0.8116
Answer Relevance	0.8750

结果说明系统能稳定检索到相关文档，且生成答案大多能被上下文支持。Answer Relevance 略低，主要来自部分问题的期望关键词和抽取句表达不完全一致。

7. Bad Case 分析

在“Redis 的 AOF 和 RDB 持久化有什么区别？”这一问题中，早期版本的检索会把 Pub/Sub 文档排到较高位置，因为 Pub/Sub 文档中也出现了“持久化”相关表述。这说明纯粹依赖关键词或粗粒度向量相似度时，技术文档中的共现词可能造成干扰。

解决方式是在生成阶段加入相对分数阈值，只使用与最高分足够接近的上下文。最终答案只引用持久化文档，避免把 Pub/Sub 内容混入持久化问题。这个案例表明 RAG 系统不仅要优化检索召回，也要控制生成上下文质量。

另一个局限是默认 hashing embedding 缺少深层语义理解，对复杂同义表达不如 BGE 等神经 embedding。为了课程复现，本项目选择轻量默认实现；实际部署可以把 embedding 模块替换为 BAAI/bge-small-zh-v1.5。

8. 结论

本文完成了一个 Redis 垂直领域 RAG 问答系统。系统具备私有知识库构建、Chunking、Embedding、向量数据库、混合检索、Query Rewriting、LLM-compatible 生成和三维量化评估能力。实验结果表明，混合检索适合 Redis 这类命令密集型技术文档问答；同时，生成阶段的上下文过滤能减少弱相关片段导致的答案污染。后续工作可扩展官方文档规模，引入 BGE 或其他神经 embedding，并使用人工标注的更大评估集进一步验证系统能力。

参考资料

1. Redis Documentation, <https://redis.io/docs/latest/>
2. Redis persistence, https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/
3. Redis Streams, <https://redis.io/docs/latest/develop/data-types/streams/>
4. Redis Cluster specification, https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/